Synthesis of Compact Flow-based Computing Circuits from Boolean Expressions

Sven Thijssen
sven.thijssen@ucf.edu
University of Central Florida
Orlando, FL, USA

Muhammed Rashedul Haq Rashed
muhammad.rashed@ucf.edu
University of Central Florida
Orlando, FL, USA

Sumit Kumar Jha
sumit.jha@fiu.edu
Florida International University
Miami, FL, USA

Rickard Ewetz
rickard.ewetz@ucf.edu
University of Central Florida
Orlando, FL, USA

ABSTRACT

Processing in-memory has the potential to accelerate high-data-rate applications beyond the limits of modern hardware. Flow-based computing is a computing paradigm for executing Boolean logic within nanoscale memory arrays by leveraging the natural flow of electric current. Previous approaches of mapping Boolean logic onto flow-based computing circuits have been constrained by their reliance on binary decision diagrams (BDDs), which translates into high area overhead. In this paper, we introduce a novel framework called FACTOR for mapping logic functions into dense flow-based computing circuits. The proposed methodology introduces Boolean connectivity graphs (BCGs) as a more versatile representation, capable of producing smaller crossbar circuits. The framework constructs concise BCGs using factorization and expression trees. Next, the BCGs are modified to be amenable for mapping to crossbar hardware. We also propose a time multiplexing strategy for sharing hardware between different Boolean functions. Compared with the state-of-the-art approach, the experimental evaluation using 14 circuits demonstrates that FACTOR reduces area, speed, and energy with 80%, 2%, and 12%, respectively, compared with the state-of-the-art synthesis method for flow-based computing.

ACM Reference Format:

Sven Thijssen, Muhammed Rashedul Haq Rashed, Sumit Kumar Jha, and Rickard Ewetz. 2024. Synthesis of Compact Flow-based Computing Circuits from Boolean Expressions. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3657340

1 INTRODUCTION

Advances in computer hardware and system have long relied on Moore's law for continuous performance improvements [10]. However, we are quickly approaching the physical limit of the minimum reliable transistor size. Moreover, today's computing systems are also constrained by the von Neumann bottleneck [15] which separates a system into two key components: the central processing unit (CPU) and the memory unit(s). The majority of data is stored in the memory unit and is shuttled to and from the CPU when a computation takes place. The limited bandwidth of the data bus between the two units has a negative effect on the overall computing performance for high-data-rate applications. This has spurred the interest in emerging computing technologies and paradigms for future computing systems. Recent investigations include quantum computing [2], photonic computing [14], and in-memory computing using non-volatile memory (NVM) [12].

Many in-memory computing paradigms using memristors have been proposed, such as Memristor Ratioed Logic (MRL) [7], Memristor-Based Material Implication (IMPLY) [8], Memristor-Aided Logic (MAGIC) [9], and flow-based computing [1]. However, MRL operates on voltage levels instead of stored memory, IMPLY suffers from being destructive, and MAGIC is mainly efficient for parallel and regular computational patterns [11]. In contrasts, flow-based computing has been shown to be efficient for executing irregular Boolean functions.

Flow-based computing is based on programming the non-volatile memory devices within a nanoscale crossbar to a specific pattern based on an input instance of a Boolean function. This layout is designed such that two wires within the crossbar will be connected by a low-resistance path if and only if the data stored in the memristors satisfy a desired logic function. This allows for near-instant execution of digital functions by applying a voltage to the top-most wordline and measuring the output current/voltage from the bottom-most wordline. While the design of these crossbar circuits can be done by hand [1], automated methods have recently been explored in [4]. A crucial design choice within these methods is the selection of the representation for the Boolean function. Techniques have been developed based on disjunctive/conjunctive normal form (DNF/CNF) [17], binary decision diagrams (BDDs) [5], and free binary decision diagrams (FBDDs) [6]. The state-of-the-art tool COMPACT [16] is based on first converting Boolean functions into BDDs, which are directly mapped into crossbar designs for flow-based computing. With respect to the provided BDD, COMPACT is optimal in terms of hardware resources. Unfortunately, the size of BDDs are known to scale poorly for some Boolean functions, which translates into high area overheads.

The authors acknowledge support from NSF awards #2319399, #2408925, and #2404036.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3657340

In this paper, we introduce FACTOR, a framework for mapping Boolean functions to area-efficient flow-based computing circuits. The framework is based on introducing Boolean connectivity graphs (BCGs), which are used to concisely represent Boolean functions. The BCGs allow a single node to have more than two outgoing edges, which leads to an inherent advantage over BDD-based approaches. Next, the BCGs are slightly modified to satisfy crossbar imposed hardware constraints, and the resulting graph is subsequently bound to a nanoscale array. While BCGs are straightforward for single-output Boolean functions, the construction of BCGs for multi-output Boolean functions is achieved using time multiplexing. The time multiplexing allows hardware reuse across different outputs, which translates to further area savings. We evaluate our proposed FACTOR framework on 14 Revlib benchmarks. Compared with the previous state-of-the-art synthesis method for flow-based computing [16], FACTOR reduces area, latency, and energy by 80%, 2%, and 12% on average.

In Section 2, we provide preliminaries on negation normal form, crossbar arrays, and flow-based computing. In Section 3, we provide motivation for our framework and in Section 4, we introduce the FACTOR framework. An extension to sharing logic in multi-output Boolean functions is provided in Section 5. Experimental evaluation is discussed in Section 6, and the paper is concluded in Section 7.

2 PRELIMINARIES

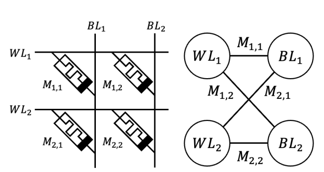
2.1 Negation Normal Form and Factoring

A Boolean expression in Negation Normal Form (NNF) is characterized by the $\neg$ (not) operation being exclusively applied to individual Boolean variables, and otherwise only uses the Boolean operators $\wedge$ (logical AND) and $\vee$ (logical OR). Any arbitrary Boolean expression can be converted into NNF form by applying De Morgan's law to any $\neg$ operation that is applied to more than a single literal. The simplification may need to be applied recursively to the subexpressions produced, until the expression is in NNF form. The NNF form is of interest because crossbars within flow-based computing only supports negation operations on the primary inputs.

Boolean functions in NNF form can be factored to reduce the literal count. Factoring involves extracting subexpressions that are common for multiple parts of a function or multi-output functions. For example, the Boolean function $f = (a \wedge c) \vee (\neg b \wedge c)$ can be factored into $f = (a \vee \neg b) \wedge c$, which reduces the literal count from four to three. Several logic synthesis tools are available for this purpose, including SIS [13].

2.2 Memristor Crossbars and Flow-based Computing

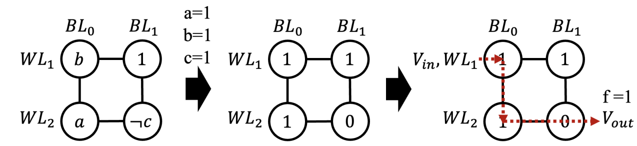
Nanoscale crossbar arrays organize memristors into a cross-point structure using wordlines and bitlines to store large amounts of data [12]. A memristor is placed at each intersection, with one end connected to the nearby wordline and the other to the nearby bitline.



(a) Memristor crossbar (b) Bipartite graph model

Figure 1: Memristor crossbar and bipartite graph model.

An example of a memristor crossbar is shown in Figure 1(a). A memristor crossbar can be modeled as a bipartite graph B where each wordline WL_i and bitline BL_j are modeled as a node. The memristor M_{ij} at their intersection is modeled as an edge between the nodes WL_i and BL_j. The graph model is shown in Figure 1(b).



(a) Crossbar design D_f (b) Programmed crossbar (c) Evaluation

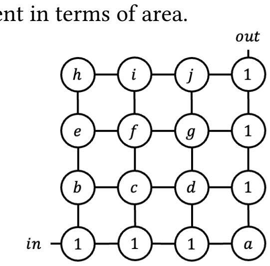
Figure 2: Example of flow-based Computing

Flow-based computing is a digital in-memory computing paradigm, executing Boolean functions using a 3-stage process on nanoscale crossbars. First, an abstract crossbar design is synthesized from a Boolean function. The design dictates where Boolean variables (or their complements) should be stored in the memristor array. A crossbar representation of the Boolean function $f = (a \wedge b) \vee \neg c$ is shown in Figure 2(a). Second, the crossbar hardware is programmed with respect to an instance of the Boolean variables. In Figure 2(b), the crossbar is programmed with respect to the instance of the Boolean variables $(a, b, c) = (1, 1, 1)$. Third, the function is evaluated by applying a positive voltage to the designated *in* wordline and measuring the voltage at the designated *out* wordline alongside a pull-down resistor. The voltage will only be high if there exists a low-resistance path between the *in* and *out* wordlines through the memristors with a stored value of 1. The function is shown to evaluate to *true* in Figure 2(c).

3 MOTIVATION

3.1 Limitations of BDD-based synthesis

There is an inherent limitation of synthesizing crossbars from BDDs. Each node in a BDD has at most 2 output edges, so the total number of edges can be no more than twice the number of nodes. On a crossbar, the nodes represent rows and columns, and the edges represent used memristors (which are not permanently set to 0 or 1). Thus—denoting the number of used memristors n—the number of rows and columns will be $\in \Theta(n)$, and the number of intersections in the crossbar will be $\in \Theta(n^2)$. For large n, this creates crossbars that are very sparsely populated by used memristors, which is not efficient in terms of area.



Area cost for N-input OR

Inputs	BDD	Optimal
1	1	1
10	100	16
100	10,000	121
10000	1,000,000	1,089

Figure 3: (a) Crossbar design for a 10-input OR, and (b) an overview of the area cost for an N-input OR using either BDDs or their optimal solutions for varying values of N.

To justify our claim, we perform a small case study in Figure 3. We illustrate an optimal crossbar design for a 10-input OR function

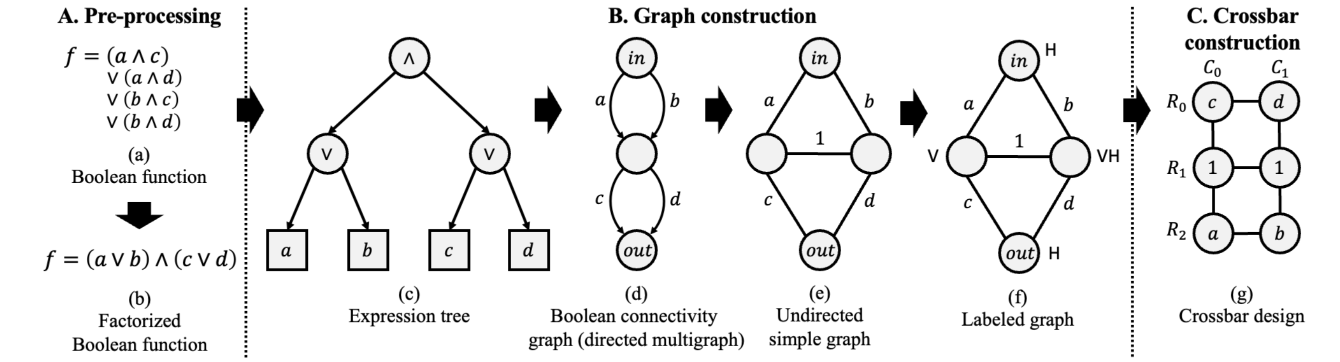


Figure 6: An example of the synthesis flow using the FACTOR framework. (a) The input is a Boolean function, which is subsequently factorized (b). Next, the factorized expression is converted into an expression tree in (c). From this tree, a Boolean connectivity graph is constructed in (d), which is a directed multigraph. Subsequently, the directed multigraph is converted into an undirected simple graph in (e), which is then labeled in (f). Finally, a crossbar design is constructed in (g).

in a crossbar with dimensions 4×4 in Figure 3(a). The equivalent layout produced by a BDD can be no smaller than 10×10. Clearly, using an alternative data structure, it is possible to construct crossbar designs which are more dense, and consequently have smaller area. In the next section, we introduce Boolean connectivity graphs, which can be used to synthesize such dense crossbar designs.

3.2 Proposed Boolean Connectivity Graphs (BCGs)

In this paper, we propose the use of Boolean Connectivity Graphs (BCGs). A BCG is a graph representation of a Boolean function f. The graph $G = (V, E)$ has a set of nodes V and a set of edges E labeled with Boolean literals $\{x_0, \neg x_0, ..., x_n, \neg x_n\}$ or Boolean truth values $\{0, 1\}$.

In contrast with BDDs, the graph is a directed multigraph where each node may have more than two outgoing edges. Binary decision diagrams (BDDs) are a subset of BCGs and are commonly used to synthesize memristor crossbars. They have certain structural restrictions that, while making them easier to map to layouts, prevent them from accessing the full solution space. In Figure 4(a), we show a BDD and in Figure 4(b), we show an equivalent BCG. The BDD has five nodes (including terminal nodes) and six edges whereas the BCG has three nodes and three edges. In our proposed synthesis method, we will leverage this alternative data structure to construct more succinct crossbar designs than with BDDs.

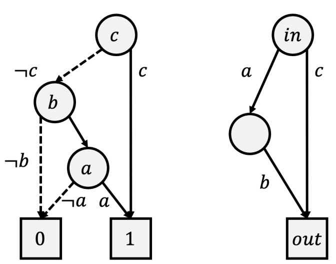


Figure 4: Comparison of graph size for a BDD and a BCG representing the same Boolean function $f = (a \wedge b) \vee c$.

4 THE FACTOR FRAMEWORK

In this section, we introduce our proposed FACTOR framework. The input to FACTOR is a single-output Boolean function f and the output is a crossbar design D_f realizing the Boolean function f. The framework consists of three main steps: (1) pre-processing, (2) graph construction, and (3) crossbar construction. A high-level overview of the FACTOR framework is shown in Figure 5. For the crossbar construction, we seek a graph representation for a one-to-one mapping between the graph and the crossbar. Based on the graph properties of a crossbar design outlined in Section 2.2, we identify two properties the graph must satisfy:

I. **Property I:** The graph must be a simple graph. This entails that for every pair of nodes, there must be at most one edge between them.
II. **Property II:** The graph must be a bipartite graph.

Another main objective of the FACTOR framework is to identify multi-input OR operations, which can be realized efficiently using the dense crossbar structure, as shown in Figure 3(a).

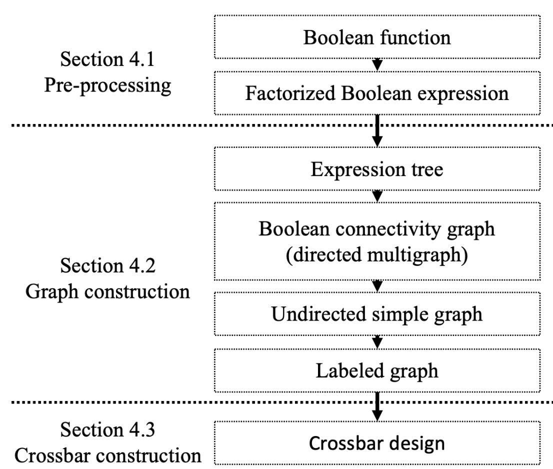


Figure 5: Overview of the FACTOR framework.

4.1 Pre-processing

The input of the first step is the Boolean function f, as shown in Figure 6(a). The Boolean function f is described in a hardware descriptive language such as Verilog or VHDL. Next, the Boolean function f is converted into NNF and factorized, as shown in Figure 6(b). For factorization, we use the logic synthesis tool SIS [13].

4.2 Graph construction

In this section, we construct a labeled graph from the given factorized Boolean function f using a Boolean connectivity graph. The graph construction has four substeps: (1) construction of an expression tree, (2) construction of a Boolean connectivity graph, (3), construction of an undirected simple graph, and (4) construction of a labeled graph.

4.2.1 Expression tree. First, we construct an expression tree from the factorized Boolean function, as shown in Figure 6(c). All nodes in the tree, except for the leaf nodes, are labeled with the Boolean operators $\wedge$ or $\vee$. The leaf nodes are labeled with Boolean literals $\{x_1, \neg x_1, ..., x_n, \neg x_n\}$. The expression tree is constructed bottom-up where the binary operators are converted into nodes according to the order of operations. These nodes have two outgoing edges, one to each operand.

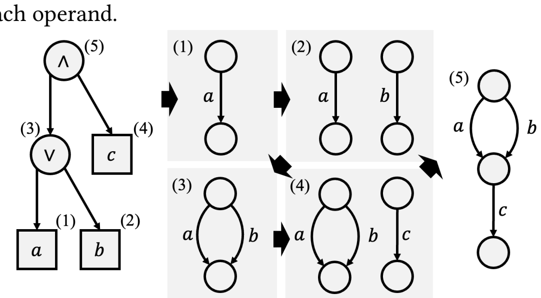


Figure 7: Construction of a Boolean connectivity graph (a directed multigraph) from an expression tree using a recursive algorithm. Each node corresponds to a recursive step and is indicated by a number between brackets, from which a BCG is subsequently constructed.

4.2.2 Boolean connectivity graph. Given the expression tree, we construct a Boolean connectivity graph using the recursive Algorithm 1. The Boolean connectivity graph is a directed multigraph where the edges are labeled with literals. The algorithm works as follows: first, it constructs the graphs for the left and right sub-trees of the expression. Then, it merges those graphs based on the top-level operation for the expression. If the operation is a conjunction ($\wedge$), then the two graphs are placed into series. Otherwise, if the operation is a disjunction ($\vee$), then the two graphs are placed in parallel. It is precisely this disjunction operation which provides the expressive power for Boolean connectivity graphs. Where binary decision diagrams have at most two outgoing edges, BCGs may have more than two. The recursion halts when the expression is empty. A detailed example of the algorithm is provided in Figure 7.

4.2.3 Undirected simple graph. Given the Boolean connectivity graph, we want to construct a new graph to satisfy Property I. The input is the Boolean connectivity graph, and the output is an undirected simple graph. The Boolean connectivity graph is a directed multigraph, which entails that there may be more than one edge between two pairs of nodes. To construct the simple graph, we will split nodes into new nodes, and connect these newly constructed nodes. More specifically, given two pairs of nodes u and v with k edges between them, we will construct $\delta(u)-1$ and $\delta(v)-1$ duplicate nodes respectively, such that $\delta(u) \times \delta(v) \geq k$. Then, the duplicate nodes and the original node are connected using an edge with Boolean truth value 1 (*true*).

Algorithm 1 Expression tree to Boolean connectivity graph

```
procedure CONVERT(expression)
    if expression.operation == None then
        return Graph(expression.literal)
    end if
    left ← Convert(expression.left)
    right ← Convert(expression.right)
    if expression.operation == ∧ then
        Merge(left.output, right.input)
        return Graph(left, right)
    else if expression.operation == ∨ then
        Merge(left.input, right.input)
        Merge(left.output, right.output)
        return Graph(left, right)
    end if
end procedure
```

In Algorithm 2, we provide an algorithm to determine the number of duplicate nodes that must be constructed for each node to satisfy the condition. Initially, we set $\delta = 1$ for all nodes. Subsequently, we iterate over all edges between every pair of nodes. The algorithm tries to evenly distribute the duplicate nodes by incrementing δ for either u or v as long as $\delta(u) \times \delta(v) < k$. In Figure 8(a), we show the values of δ for three nodes, in (b) the duplication of nodes, and in (c) the resulting simple graph.

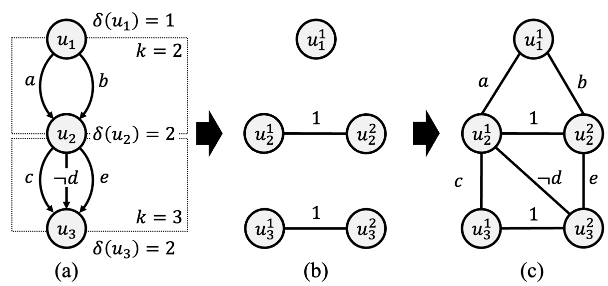


Figure 8: Construction of a simple undirected graph from a directed multigraph.

Algorithm 2 Computing the number of duplicate nodes

```
for node in graph.nodes do
    δ(node) = 1
end for
for node1 in graph do
    for node2 in graph do
        k ← graph.CountEdges(node1, node2)
        while δ(node1) × δ(node2) < k do
            if δ(node1) < δ(node2) then
                δ(node1) ← δ(node1) +1
            else
                δ(node2) ← δ(node2) +1
            end if
        end while
    end for
end for
```

Table 1: Comparison of the graph properties (nodes and edges) and the hardware resources for the crossbar design (rows, columns, semiperimeter, area, and number of non-zero memristors) for both multiple single-output Boolean functions and a single multi-output Boolean function.

Benchmark	Multiple single-output Boolean functions								Single multi-output Boolean function							
	Graph		Crossbar design						Graph		Crossbar design					
	Nodes (num)	Edges (num)	Rows (num)	Cols (num)	Semi (num)	Area (num)	Mems (num)	Time (s)	Nodes (num)	Edges (num)	Rows (num)	Cols (num)	Semi (num)	Area (num)	Mems (num)	Time (s)
cm150a	32	47	22	27	49	594	47	0.34	33	48	12	21	33	252	48	0.019
t481	44	70	23	31	54	713	64	0.265	45	71	31	25	56	775	65	0.153
x2	52	66	24	36	60	864	60	0.893	36	62	19	27	46	513	58	0.016
cm163a	77	88	33	48	81	1584	73	0.841	44	73	24	33	57	792	68	0.017
misex1	76	96	37	47	84	1739	92	0.623	37	66	23	26	49	598	61	0.02
cordic	97	152	63	58	121	3654	145	0.343	97	154	40	70	110	2800	147	36.8
5xp1	124	175	61	84	145	5124	161	1.09	105	164	56	83	139	4648	160	0.024
clip	159	251	89	112	201	9968	231	0.531	165	250	93	116	209	10788	241	0.029
alu4	1189	1706	648	800	1448	518400	1649	15.336	1296	1834	697	863	1560	601511	1813	3.12
misex3	1245	1864	636	828	1464	526608	1812	8.637	752	1220	418	540	958	225720	1194	0.89
apex2	390	571	217	257	474	55769	546	39.471	415	599	227	280	507	63560	578	1.27
apex4	4206	5525	2241	2797	5038	6268077	5479	8.2	1402	3120	813	1030	1843	837390	2840	5.22
apex5	2117	3003	1136	1302	2438	1479072	2652	11.383	1851	2911	964	1198	2162	1154872	2740	0.117
seq	2733	3665	1408	1742	3150	2452736	3557	37.11	1459	2202	749	1016	1765	760984	2165	0.539
Geomean	271.0	380.9	143.9	180.9	325.2	26017.6	357.2	2.3	204.5	328.8	108.6	141.9	251.5	15402.9	313.8	0.2
Ratio	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	**0.8**	**0.9**	**0.8**	**0.8**	**0.8**	**0.6**	**0.9**	**0.1**

4.2.4 Node labeling. To satisfy Property II from Section 4, we must convert the undirected simple graph into an undirected bipartite graph. In previous work, a labeling algorithm was proposed where nodes are labeled with V, H, or VH, indicating whether they will be assigned to vertical, horizontal, or both vertical and horizontal nanowires in the crossbar design [16]. An integer linear programming solution (ILP) was proposed which is NP-hard and thus time-consuming. We employ a faster and more practical greedy algorithm to label the nodes with V, H, or VH based on the constraints in [16].

4.3 Crossbar construction

The final step is the construction of a crossbar design based on the labeled undirected simple graph. In [16], an analogy was defined between a the graph of a binary decision diagram and a memristor crossbar. In this analogy, the nodes are assigned to the wordlines and bitlines in a crossbar according to their label V, H, or VH. The edges with the literals as labels are assigned to memristors in a crossbar. For this last step, we use the same analogy to construct the crossbar design using the labeled undirected simple graph.

5 MULTI-OUTPUT BOOLEAN FUNCTIONS

Real-world combinational functions typically have multiple outputs. Sharing logic among multiple Boolean functions may result in smaller crossbar designs. Therefore, we propose a multi-step evaluation methodology based on multiplexing for multi-output Boolean functions. More specifically, for each output function f_i, we introduce a selector variable s_i. Then the overall Boolean function is constructed as a disjunction of the selector variable s_i conjoined with the function f_i, as shown below:

$$f_1 = (a \vee b) \wedge c \tag{1}$$

$$f_2 = a \vee b \tag{2}$$

$$f = (s_1 \wedge f_1) \vee (s_2 \wedge f_2) \tag{3}$$

The Boolean function f (line 3) then follows the same synthesis method as in Section 4. Evaluation of the multi-output Boolean function with N outputs is then accomplished using N steps. In each step $i, i \in [1, .., N]$, only the selector variables s_i are set to 1 while the other selector variables $s_j, j \neq i$, are set to 0. In Figure 9, we illustrate the evaluation of a multi-output Boolean function.

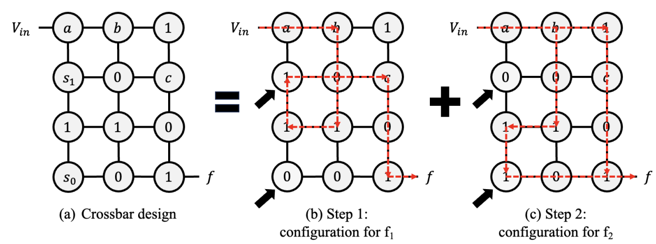


Figure 9: Multi-step evaluation for a multi-output Boolean function with outputs f_1 and f_2. For each step, we have highlighted the paths for the respectively Boolean functions using dashed lines in red. The paths for both outputs are different due to the different state of the memristors with variables s_1 and s_2. For f_1, the selector variables are set to $s_1 = 1$ and $s_2 = 0$, and for f_2 to $s_1 = 0$ and $s_2 = 1$.

6 EXPERIMENTAL RESULTS

In this section, we will evaluate our proposed FACTOR framework and compare the framework with other digital in-memory computing paradigms. The code for FACTOR is written in Python and the experiments are conducted on a machine with a 12th Gen Intel® Core™ i7-12700 × 20 processor. The code is on GitHub[1]. The experiments are conducted on 14 benchmarks from RevLib [18].

6.1 Evaluation of the FACTOR framework

First, we evaluate the proposed FACTOR framework in terms of graph properties and hardware resources. The graph properties are the number of nodes and edges for the Boolean connectivity graph. The hardware resources are the number of rows, columns, semiperimeter (rows+columns), area (rows×columns), the number of non-zero memristors, and the synthesis time. In Table 1, we compare the graph properties and hardware resources for the crossbar design for two approaches. In the first approach, the benchmarks are synthesized as multiple single-output Boolean functions and in the second approach as a single multi-output Boolean function. We observe that the number of nodes and the number of edges is reduced by 20% and 10% when the logic is shared. Consequently,

[1] https://github.com/sventhijssen/factor

Table 2: Comparison of the hardware resources in terms of rows, columns, semiperimeter, area, non-zero memristors, and synthesis time for different synthesis methods for flow-based computing.

Benchmark	Chakraborty et al. [4]						COMPACT [16]						FACTOR					
	Rows (num)	Cols (num)	Semi (num)	Area (num)	Mems (num)	Time (min)	Rows (num)	Cols (num)	Semi (num)	Area (num)	Mems (num)	Time (min)	Rows (num)	Cols (num)	Semi (num)	Area (num)	Mems (num)	Time (min)
cm150a	32	48	80	1536	48	0.0	12	22	34	264	48	0.0	22	27	49	594	47	0.0
t481	32	37	69	1184	58	0.0	17	23	40	391	58	0.0	23	31	54	713	64	0.0
x2	52	74	126	3848	89	0.0	33	35	68	1155	89	0.1	19	27	46	513	58	0.0
cm163a	66	90	156	5940	92	0.1	25	31	56	775	78	0.0	24	33	57	792	68	0.0
misex1	72	92	164	6624	101	0.0	21	29	50	609	72	0.0	23	26	49	598	61	0.0
cordic	99	131	230	12969	170	0.0	42	44	86	1848	142	0.1	63	58	121	3654	145	0.0
5xp1	117	155	272	18135	185	0.1	52	53	105	2756	162	0.1	56	83	139	4648	160	0.0
clip	148	186	334	27528	253	0.0	84	84	168	7056	253	0.0	89	112	201	9968	231	0.0
alu4	1281	1461	2742	1871541	2299	0.4	683	686	1369	468538	2299	1.5	648	800	1448	518400	1649	0.3
misex3	1552	1670	3222	2591840	2486	0.4	674	676	1350	455624	2292	2.2	418	540	958	225720	1194	0.0
apex2	1644	1645	3289	2704380	2759	1.4	909	936	1845	850824	2759	1.1	217	257	474	55769	546	0.7
apex4	1644	1789	3433	2941116	2912	0.4	508	528	1036	268224	1910	2.3	813	1030	1843	837390	2840	0.1
apex5	2674	3591	6265	9602334	4352	1.0	1409	1497	2906	2109273	4352	0.9	1136	1302	2438	1479072	2652	0.2
seq	3231	3690	6921	11922390	4982	1.5	1743	1778	3521	3099054	4982	1.1	749	1016	1765	760984	2165	0.0
Geomean	282.0	348.6	631.6	98307.5	458.7	0.0	128.6	145.5	275.1	18709.5	417.4	0.0	114.6	143.6	258.6	16459.9	307.6	0.0
Ratio	1.0	1.0	1.0	1.0	1.0	**1.0**	0.5	0.4	0.4	0.2	0.9	2.6	**0.4**	**0.4**	**0.4**	**0.2**	**0.7**	1.8

the required hardware resources are also lower on average with an average reduction of 20% for the semiperimeter, 40% for the area, and 10% for the number of non-zero memristors.

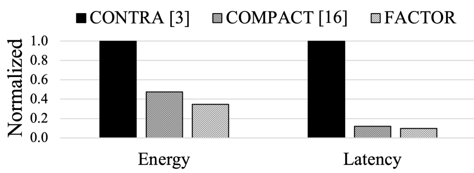


Figure 10: Comparison of average energy consumption and latency over fourteen Revlib benchmarks for CONTRA [3], COMPACT [16], and our proposed framework FACTOR.

6.2 Comparison for digital in-memory computing

In this section, we compare our proposed FACTOR framework with other frameworks for digital in-memory computing. First, we compare with previous work for flow-based computing, and then we compare with other digital in-memory computing paradigms.

In Table 2, we compare our proposed synthesis method with the previous state-of-the-art synthesis methods for flow-based computing [4, 16]. For FACTOR, we use the results with the least number of non-zero memristors from Table 1. We observe that our proposed method uses the least amount of resources with a reduction of 60% for the semiperimeter, 80% for the area, and 30% for the number of non-zero memristors. This is due to that previous work relies on BDDs as underlying data structure whereas FACTOR relies on Boolean connectivity graphs, which are more expressive.

Next, we evaluate the energy and latency for our proposed FACTOR framework for flow-based computing with CONTRA [3] for the MAGIC computing paradigm, and with COMPACT for flow-based computing. The READ/WRITE energy to program a ReRAM device is $1.08pJ$ and $3910pJ$, and the READ/WRITE latency is $29.31ns$ and $50.88ns$ [19]. In Figure 10, we show the normalized energy and latency over the 14 benchmarks for all three frameworks. We observe that our proposed FACTOR framework outperforms CONTRA and COMPACT in terms of energy and latency with an energy and latency reduction of 65% and 90% compared with CONTRA, and 12% and 2% with COMPACT. This is due to the low number of WRITE operations.

7 CONCLUSION AND FUTURE WORK

In this paper, we have presented FACTOR, a framework for synthesizing crossbar designs for flow-based computing. Our innovation lies in the exploration of data structures, called Boolean connectivity graphs, which are more succinct than binary decision diagrams. In contrasts with BDDs, BCGs are not limited to two outgoing edges for each internal node in the graph. This has a first-order impact on the overall energy efficiency and latency for the flow-based computing system. Using FACTOR, the semiperimeter (rows+columns) reduces by 60% and the number of non-zero memristors by 30% compared with the state-of-the-art. This results in an area, energy and latency reduction of 80% and 12%, and 2% compared with the state-of-the-art. In our future work, we will explore other computational structures that can supports flow-based computing systems.

REFERENCES

[1] Zahiruddin Alamgir et al. 2016. Flow-based computing on nanoscale crossbars: Design and implementation of full adders. In *ISCAS*. IEEE, 1870–1873.

[2] Frank Arute et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574, 7779 (2019), 505–510.

[3] Debjyoti Bhattacharjee et al. 2020. Contra: area-constrained technology mapping framework for memristive memory processing unit. In *ICCAD*. 1–9.

[4] Dwaipayan Chakraborty and Sumit Jha. 2017. Automated synthesis of compact crossbars for sneak-path based in-memory computing. In *DATE*. IEEE, 770–775.

[5] Amad Ul Hassen. 2017. Automated synthesis of compact multiplier circuits for in-memory computing using ROBDDs. In *NANOARCH*. IEEE, 141–146.

[6] Amad Ul Hassen et al. 2018. Free binary decision diagram-based synthesis of compact crossbars for in-memory computing. *TCAS-II* 65, 5 (2018), 622–626.

[7] Shahar Kvatinsky et al. 2012. MRL—Memristor ratioed logic. In *International Workshop on Cellular Nanoscale Networks and their Applications*. IEEE, 1–6.

[8] Shahar Kvatinsky et al. 2013. Memristor-based material implication (IMPLY) logic: Design principles and methodologies. *IEEE VLSI* 22, 10 (2013), 2054–2066.

[9] Shahar Kvatinsky et al. 2014. MAGIC—Memristor-aided logic. *IEEE TCAS-II* 61, 11 (2014), 895–899.

[10] Gordon E. Moore. 1964. Cramming More Components onto Integrated Circuits. *Electronics* (1964).

[11] Muhammad Rashedul Haq Rashed, Sumit Kumar Jha, Fan Yao, and Rickard Ewetz. 2022. Hybrid digital-digital in-memory computing. In *DATE*. IEEE, 1177–1180.

[12] Abu Sebastian et al. 2020. Memory devices and applications for in-memory computing. *Nature nanotechnology* 15, 7 (2020), 529–544.

[13] E.M. Sentovich et al. 1992. *SIS: A System for Sequential Circuit Synthesis.* Technical Report UCB/ERL M92/41. EECS Department, University of California, Berkeley. http://www2.eecs.berkeley.edu/Pubs/TechRpts/1992/2010.html

[14] Bhavin J Shastri et al. 2021. Photonics for artificial intelligence and neuromorphic computing. *Nature Photonics* 15, 2 (2021), 102–114.

[15] Thomas N Theis and H-S Philip Wong. 2017. The end of moore's law: A new beginning for information technology. *Computing in science & engineering* 19, 2 (2017), 41–50.

[16] Sven Thijssen, Sumit Jha, and Rickard Ewetz. 2021. Compact: Flow-based computing on nanoscale crossbars with minimal semiperimeter. In *DATE*. IEEE, 232–237.

[17] Alvaro Velasquez and Sumit Kumar Jha. 2014. Parallel computing using memristive crossbar networks: Nullifying the processor-memory bottleneck. In *IDT*. IEEE, 147–152.

[18] Robert Wille et al. 2008. RevLib: An online resource for reversible functions and reversible circuits. In *ISMVL*. IEEE, 220–225.

[19] Tao Yang et al. 2021. PIMGCN: a reram-based PIM design for graph convolutional network acceleration. In *DAC*. IEEE, 583–588.

Oltron: Software-Hardware Co-design for Outlier-Aware Quantization of LLMs with Inter-/Intra-Layer Adaptation

Chenhao Xue[1,4], Chen Zhang[2*], Xun Jiang[1], ZhuTianYa Gao[2], Yibo Lin[1,3,4], Guangyu Sun[1,3,4*]
[1]School of Integrated Circuits, Peking University. [2]Shanghai Jiao Tong University. [3]Institute of Electronic Design Automation, Peking University, Wuxi, China. [4]Beijing Advanced Innovation Center for Integrated Circuits.

Abstract

In Large Language Models (LLMs), outliers are identified by a small number of values with exceptionally high magnitudes, critically affecting model accuracy. Researchers have proposed several mixed-precision quantization techniques to manage these activation outliers. These approaches, employing value-wise outlier granularity, face challenges in balancing model accuracy with hardware efficiency. To address this issue, we capitalize on the observation that activation outliers of LLMs typically cluster within specific channels. Consequently, we introduce `Oltron`, a comprehensive software/hardware co-design strategy for outlier-aware quantization of LLMs with inter-/intra-layer adaptation. Our method includes three key innovations: firstly, a novel quantization algorithm that identifies the optimal ratio of outliers across different layers and channel groups within a layer; secondly, a reconfigurable architecture that adapts to inter- and intra-layer distributions; and thirdly, a tile-based dataflow optimizer that intricately arranges complex computations and memory access for mixed-precision tensors. `Oltron` outperforms the state-of-the-art outlier-aware accelerator, OliVe, achieving a 1.9x performance boost and 1.6x greater energy efficiency, while also enhancing model accuracy.

Keywords: Large Language Models, Quantization, Accelerators

ACM Reference Format:
Chenhao Xue[1,4], Chen Zhang[2*], Xun Jiang[1], ZhuTianYa Gao[2], Yibo Lin[1,3,4], Guangyu Sun[1,3,4*]. 2024. Oltron: Software-Hardware Co-design for Outlier-Aware Quantization of LLMs with Inter-/Intra-Layer Adaptation. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3656221

1 Introduction

Transformer-based large language models (LLMs) have demonstrated great success [17]. However, their remarkable performance comes with exceptional model size and computational requirements. For instance, deploying the GPT-175B model for inference in FP16 consumes at least 350GB of storage, which requires at least 5 × 80GB A100 GPUs [1]. Quantization [2, 4, 5, 13, 18, 20–22] is one of the most effective ways to reduce the inference cost of LLMs. By quantizing the weights and activations into low bit-width, the demands for both memory size and bandwidth can be greatly reduced, and compute-intensive operations can be accelerated.

*The corresponding authors: Chen Zhang, chenzhang.sjtu@sjtu.edu.cn; Guangyu Sun, gsun@pku.edu.cn.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3656221

Table 1. Comparison between outlier-aware accelerators and our proposed method `Oltron`.

Architecture		OLAccel [10]	OliVe [5]	`Oltron` **(Ours)**
Outlier Granularity		Value	Value	Channel
Accuracy	Outlier Bit-Width	16	4	8/12/16
Efficiency	Avg. Index Bit-Width	16	4	≈ 0
	Aligned Memory Access	✗	✓	✓
	Outlier PE Ratio	1/16	1	1/8
	Overhead Area Ratio	71%	0.2%	0.1%

Currently, the weights of LLMs can be quantized to 4-bit with minimal impact on model accuracy [3]. However, activation quantization of LLMs remains challenging, primarily due to outlier issues [2, 21]. The small fraction of activation values feature extremely large magnitudes and predominant impacts on model accuracy, which makes it difficult to apply low bit-width (4-bit) activation quantization without sacrificing model accuracy [13, 18, 20–22].

To maintain model accuracy, previous works have proposed outlier-aware quantization with dedicated hardware support, such as OLAccel [10] and GOBO [23]. These methods employ mixed-precision quantization, where low-bit (4-bit) quantization is applied to the majority of normal values, while high precision (16-bit) is reserved for the outliers. This approach achieves a nearly average 4-bit quantization with negligible accuracy loss. However, they treat outliers as sparse matrices, requiring additional codebooks for sparse format and complex control logic to handle irregular computations and memory accesses. OliVe [5], on the other hand, introduces outlier-victim pair (OVP) encoding with localized outlier storage, ensuring aligned memory access with minimal control overhead. Nevertheless, the localized outlier encoding scheme has limitations in extending the bit-width for outliers, resulting in insufficient precision in LLMs. Moreover, to locally process outlier values with random occurrence, OliVe augments all computing units with the capability to handle outlier values, which compromises hardware efficiency.

The aforementioned outlier-aware accelerators handle outliers at value-wise granularity, and struggle to balance model accuracy and hardware efficiency. In this work, we aim to strike such balance by handling activation outliers of LLMs at channel granularity. We leverage the observation that activation outliers are not distributed in complete randomness, but instead tend to cluster in specific channels [2, 20]. By handling outliers at coarser granularity, we can still maintain model accuracy with low average bit-width and efficient hardware design.

We introduce `Oltron`, a holistic quantization framework that encompasses algorithm-software-hardware co-optimization to effectively handle outlier activation channels while simultaneously ensuring hardware efficiency. We commence by conducting a thorough analysis of outlier distribution patterns, shedding light on the channel-wise non-uniform distribution with intra-/inter-layer heterogeneity. To tackle intra-layer heterogeneity, we introduce the *Tile-wise Outlier-Balanced Encoding* (TOBE) format. TOBE consistently guarantees dataflow regularity and incurs minimal encoding costs through compilation-time optimizations. To address inter-layer heterogeneity, we co-design TOBE with an adaptive quantization algorithm and a reconfigurable architecture, which enables flexible adjustment of TOBE settings to meet the precision requirements of each layer, while allowing for efficient TOBE acceleration. Our evaluation results show that `Oltron` achieves superior excels in both quantized model accuracy and hardware efficiency. Compared to state-of-the-art outlier-aware accelerators, `Oltron`'s accelerator design surpasses existing outlier-aware accelerators, OLAccel [10] and OliVe [5], by 3.6× and 1.9× performance improvement, and 1.9× and 1.6× energy reduction, respectively.

The main contributions of this paper are as follows:

- We introduce Tile-wise Outlier-Balanced Encoding (TOBE) format for LLM activation quantization, which encodes outlier channels with high precision while consistently maintaining dataflow regularity.
- We propose `Oltron`, a TOBE-based quantization framework integrated with software-hardware co-optimization, to fully harness the adaptability of TOBE to the inter-/intra-layer heterogeneity of outlier channel distribution.
- We propose efficient implementation of `Oltron`'s reconfigurable architecture complemented by compilation & algorithm support, and demonstrate that the accuracy and efficiency of `Oltron` outperform existing outlier-aware quantization algorithms and hardware accelerators.

2 Background & Motivation

In this section, we first review previous works dealing with the activation outlier challenge of LLMs. Our proposed channel-wise outlier-aware quantization aims to balance accuracy with efficiency. However, given the non-uniform distribution of outlier channels detailed in Sec. 2.2, it remains challenging to maintain regular computation and memory access, and thereby, to provide hardware efficiency. To tackle this challenge, we introduce a software-hardware co-design solution termed as `Oltron`, and present its framework overview in Sec. 2.3.

2.1 Related Works

Previous works [5, 10, 13, 18, 20, 23] propose various quantization/architecture co-design methods to tackle the activation outlier challenge of LLMs. Some quantization algorithms [13, 18, 20] smooth out activation distribution by scaling down outlier magnitudes, so that the tensor can be uniformly quantized. However, encoding outlier values with the same low bit-width as normal values cannot provide enough precision, consequently leading to degraded model accuracy. In addition, some outlier-aware architectures [5, 10, 23] are proposed to encode outliers with high precision and maintain model accuracy. For instance, OLAccel [10] and GOBO [23] store high-precision outliers separately from normal values, and employ dedicated processing units for the outliers. However, the outlier storage has variable length due to random occurrence of outliers, which results in unaligned memory access, Moreover,

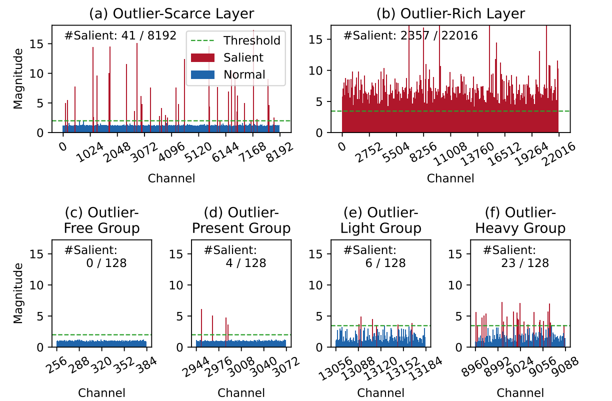


Figure 1. Input activation distribution of linear layers from LLaMA-65B. For illustration, channels of > 2× median magnitude are deemed *salient*. (a) and (b) are high-level views resized with max-pooling. (c-d) / (e-f) are zoom-in views of (a) / (b), respectively.

the separated outlier storage requires complex control logic to index outliers and orchestrate the computation between normal & outlier values. In contrast, OliVe [5] locally stores outlier values, and allocates co-located bit-width for indexing. OliVe manages to achieve aligned memory access and negligible control overhead. However, it can only extend limited bit-width to represent outlier values, which also suffers from insufficient precision and model accuracy loss. Moreover, in accommodation to outlier values with local storage and random occurrence, OliVe extends all computing units to enable outlier processing, which adversely impacts its hardware efficiency.

2.2 Outlier Distribution Characteristics

Fig. 1 visualizes the input activation distribution of two representative linear layers in the LLaMA-65B model. We randomly sampled 128 sequences from Wikitext2 dataset [8]. For every input channel, we recorded the maximum absolute value across all tokens. From these figures, we observe some intriguing characteristics of outlier distribution, namely: the presence of salient channels, and the inter-/intra-layer heterogeneity of outlier distribution.

Salient Channels refers to the ones with significantly larger magnitudes than most of other channels. The saliency of these channels is reflected in two aspects. Firstly, they tend to have larger quantization errors than normal channels under the same bit-width. Secondly, quantizing salient channels alongside normal channels results in either clamping of the former or substantial rounding errors in the latter, both resulting in model accuracy losses.

Inter-Layer Heterogeneity refers to the observation that different layers exhibit significantly different composition ratios of salient channels. For instance, as shown in Fig. 1(b), the MLP contraction layer contains a considerably higher number of outlier channels compared to the attention input projection layer depicted in Fig. 1(a). Therefore, we categorize the former as an *outlier-rich* layer and the latter as an *outlier-scarce* layer.

Intra-Layer Heterogeneity refers to the observation that salient channels are randomly distributed within a layer, resulting in various salient channel proportions across channel groups. In Fig. 1(c-f), significant distribution variances are evident among four distinct channel groups from the two selected layers, each comprising 128 channels. Some groups, such as those in Fig. 1(c), are entirely devoid of outliers, while others, like those in Fig. 1(f), exhibit a larger

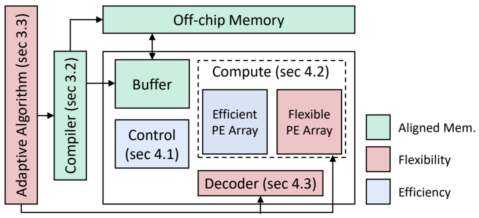


Figure 2. `Oltron`'s framework overview.

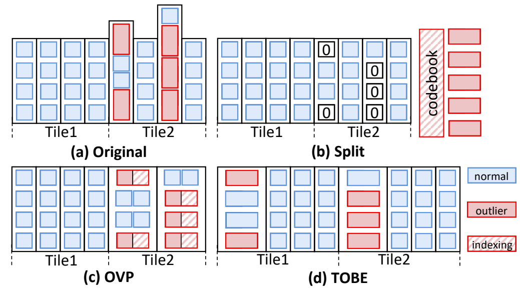


Figure 3. Comparisons between different encoding format.

proportion of salient channels, with the remaining groups falling somewhere in between.

2.3 Framework Overview

To address the efficiency challenges arised from inter-/intra-layer heterogeneity, we introduce a novel representation format called *Tile-wise Outlier-Balanced Encoding* (TOBE). TOBE supports encoding salient channels with arbitrary proportion and unrestricted precision, while maintaining storage regularity. To efficiently harness the flexible expressiveness of TOBE, we introduce `Oltron`, a versatile and reconfigurable quantization framework empowered with software-hardware co-optimization. As for software, `Oltron` utilizes an adaptive quantization algorithm to reduce mixed-precision overhead without compromising encoding precision, and leverages dedicated compilation support to achieve negligible encoding overhead. As for hardware, `Oltron` adopts an efficient accelerator design to support TOBE-based dataflow, with runtime reconfigurability to accommodate TOBE with various salient channel proportion & precision settings. The specific contents of the framework and their corresponding chapters are summarized in the Fig. 2

3 Software Design

In this section, we first present details of TOBE that balances the distribution of outlier channels. TOBE features the capability to keep computation and memory access regular without sacrificing outlier's high precision encoding. Since operators are connected through tensors, i.e. one operator's output is another operator's input, we propose a TOBE-aware dataflow optimization that optimizes the memory-related layout conversion operations across the whole computation graph. Finally, based on TOBE, we propose an adaptive quantization algorithm that automatically selects salient channel configurations to determine the best trade-off between mixed-precision overhead and model accuracy.

3.1 Tile-wise Outlier-Balanced Encoding

To execute large tensor operators in LLMs, tiling is a must to split computations and associated input tensors into sub-blocks, allowing efficient caching in on-chip memory. However, mixed-precision quantization may not integrate well with the tiling approach. To encode the mixed-precision tensor, a straightforward approach is to store outliers in their original positions (Fig. 3(a)). As outliers require additional encoding bit-width, this method results in irregular tensor storage and inconsistent data tile sizes. Such imbalance leads to unaligned memory access and under-utilization of on-chip resources, and ultimately reduced achievable performance. The split scheme [10, 23] addresses the memory alignment challenge by separating sparse outlier encoding to ensure uniformity of normal tiles (Fig. 3(b)), yet the problem persists for the variable-length outlier encoding. It also introduces complicated control overhead to manage the separately stored and computed normal/outlier values. To achieve memory alignment with reduced control overhead, the outlier-victim pair encoding [5] improves upon the original scheme. It prunes the victim values adjacent to the outliers, and allocates the saved bit-width for extended outlier representation (Fig. 3(c)). However, this approach only offers limited bit-width expansion for outliers, resulting in constrained precision of outlier representation, and ultimately impacting the model's accuracy.

TOBE achieves regular memory access pattern by adjusting precision at channel granularity, and reordering high precision channels to ensure balanced sizes of tiled sub-blocks, which greatly reduces the complicated control logic required for the sparse outlier values (Fig. 3(d)). As opposed to handling outlier values individually in previous encoding schemes, TOBE encodes salient channels with severe outlier issues at high precision. Regarding the observed intra-layer random distribution of salient channels (Fig. 1), to achieve balanced tile sizes, TOBE evenly distributes salient channels into tiled sub-blocks by reordering the channel permutation. Since on-chip buffer accesses off-chip memory in units of data tiles, TOBE's uniform data tile sizes ensure regular off-chip memory access. In addition, within each tiled sub-block, the assigned salient channels are placed at the forefront. The determined and consistent data layout of tiled sub-blocks ensures that on-chip memory access and computation can proceed in a regular manner.

`Oltron` mainly applies TOBE on input activation tensors of linear layers that exhibit significant outlier problems. To guarantee the correctness of the matrix multiplication results, given the reordered channel permutations of activation tensors, we also need to reorder the input channels of weight tensors correspondingly, as shown in Fig. 4(a). As the salient channels are selected based on their extreme magnitudes on calibration data, the channel permutations are predetermined. Therefore the rows of weight tensors can be permuted statically before deployment.

3.2 Dataflow Optimization

TOBE reorders the tensor layout tile-wise in memory to enhance intra-layer computation efficiency. However, this encoding necessitates an explicit reordering operation to prepare the data because the output of the previous operator is not inherently encoded in TOBE, as shown in Fig. 4. To mitigate the overhead introduced by the reordering operation, we propose dataflow optimization with two strategies applied to the computation graph, indicated by circled numbers ❶-❹ in Fig. 4.

First, we leverage the inherent commutative characteristics in matrix multiplications. In a $\langle M \times N \times K \rangle$ matrix multiplication, the reordering operation can be directly applied to the previous layer's output. For example, reordering rows in the M dimension or columns in the N dimension does not influence the final results. Moreover, if the layers between the two matrix multiplication layers also exhibit commutative characteristics, such as element-wise operators (activation layers), this reordering strategy can propagate

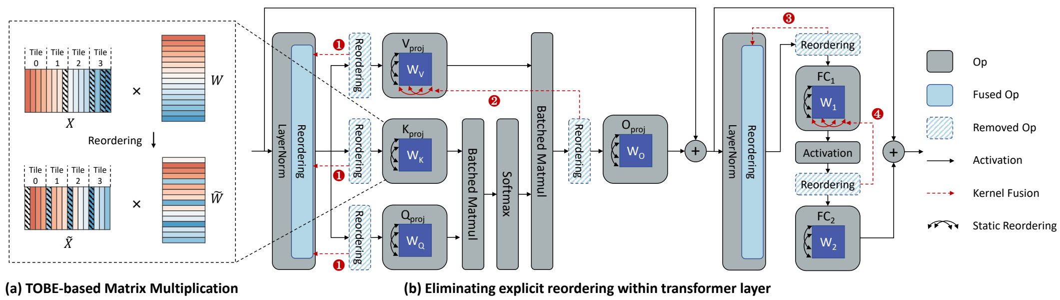


Figure 4. TOBE-based computation workflow. (a) Linear layers adopt TOBE-based matrix multiplication. (b) The encoding overhead of explicit run-time reordering is mitigated with graph-level compilation-time optimization.

Algorithm 1 Adaptive Quantization Algorithm

Input: activation statistics $\mathcal{M} = \{M \in \mathbb{R}^{\#\text{batch} \times d_l} | 1 \leq l \leq L\}$, storage budget $B^* \in \mathbb{R}$;
Output: salient channel sets $\mathcal{S} = \{S_l | S_l \subseteq \{1, 2, \cdots d_l\}, 1 \leq l \leq L\}$, layer-wise salient channel data types $\vec{t} \in \mathcal{T}^L$;

for layer $l \in \{1, 2, \cdots, L\}$ **do**
  $\tau_l \leftarrow 3 \times$ Standard-Deviation(M_l)
  $t_l \leftarrow$ FP8
while target budget B^* not reached **do**
  $\langle e, B \rangle \leftarrow$ Estimate-MSE-And-Budget($\mathcal{M}, \vec{\tau}, \vec{t}$)
  $i \leftarrow -\infty$
  $T \leftarrow$ Considered-Modification($\vec{\tau}, \vec{t}$)
  for $\langle \vec{\tau}', \vec{t}' \rangle \in T$ **do**
    $\langle e', B' \rangle \leftarrow$ Estimate-MSE-And-Budget($\mathcal{M}, \vec{\tau}, \vec{t}$)
    $i' \leftarrow$ Estimate-Improvement($e - e', B - B'$)
    if $i' > i$ **then**
      $\langle i, \vec{\tau}, \vec{t} \rangle \leftarrow \langle i', \vec{\tau}', \vec{t}' \rangle$
$\mathcal{S} \leftarrow$ Select-Salient-Channels($\mathcal{M}, \vec{\tau}$)
Return $\mathcal{S}, \vec{t}$

through them. Two cases, layers Q_{proj} (❷) and FC_2 (❹), illustrate situations where we can eliminate explicit reordering operators by statically permuting the columns of previous layers' weight tensors.

Second, we use a kernel fusion method to merge the reordering operation with the previous layer. For $Q_{proj}/K_{proj}/V_{proj}$ (❶) and FC_1 (❸), we fuse them into previous LayerNorm operators, where each output channel is redirected to a different address in the off-chip memory to match the TOBE format of the next layer's output.

We implement this dataflow optimization in the framework's compilation pass. We traverse the entire computation graph, examining every pair of linear layers (e.g., FC1 and FC2). We also analyze the operators directly connected between them (e.g., activation layers) and apply the two aforementioned strategies to eliminate any additional memory access caused by explicit reordering operations.

3.3 Adaptive Quantization Algorithm

`Oltron` adopts mixed-precision quantization at the granularity of channels. In this approach, data within each channel is encoded in either low-precision or high-precision format. Normal channels use a 4-bit uniform encoding. For salient channels, we allow the algorithm to automatically select from multiple high-precision data type candidates, including `fp8`, `fp12`, and `fp16`.

Our algorithm automatically determines two key factors: (1) the composition ratio of salient channels relative to normal channels and (2) the most suitable bit-width for salient channels. It begins with an initial budget parameter, denoted as $\langle B^* \in \mathbb{R} \rangle$, and a set of parameters $\langle \vec{\tau} \in \mathbb{R}^L, \vec{t} \in \mathcal{T}^L \rangle$. The budget B represents the target averaged bit-width of the mixed-precision models. We use $\vec{\tau}$ to classify values as outliers or normal values, which is initialized based on an empirical 3σ rule [19], and $\vec{t}$ to specify the data types

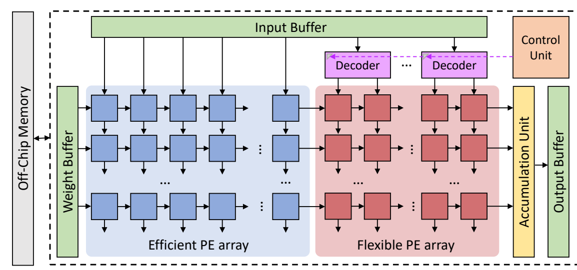


Figure 5. Overview of `Oltron` architecture.

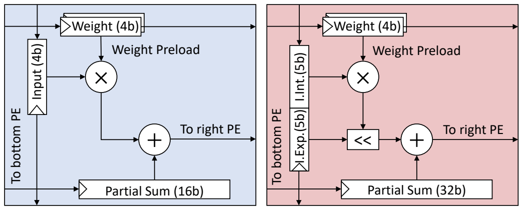
to be used for high-precision channels. With these inputs, our algorithm tunes the $\langle \vec{\tau}, \vec{t} \rangle$ parameters to control the ratios of outliers to normal values in each layer. In each iteration, we determine the remaining budget by comparing the current average bit-width B with the target budget B^*. Based on this comparison, we either assign more channels as outliers or recall outliers back. We determine the changes to the outlier ratio in each layer by calculating the mean squared error (MSE) e between the original and post-change values. The algorithm continues iterating until it reaches the target bit-width B^*.

4 Architecture Design

In this section, we present `Oltron`'s efficient and reconfigurable architecture. Our hardware design efficiently supports TOBE-based dataflow by leveraging the regularity merit of TOBE, and incorporates reconfigurable components to flexibly accommodate various TOBE settings.

4.1 Architecture Overview

Fig. 5 presents the overview of `Oltron`'s architecture. It consists of a weight-stationary systolic array with hybrid PE designs, on-chip buffers for input/weight/output, an accumulation unit, a control unit, and decoders. `Oltron` architecture adopts a 4-stage pipeline similar to Google's TPU [6]: **(1) Reading Off-chip Memory:** The weight/input buffer loads outlier-balanced data tiles from off-chip memory. **(2) Preloading:** Given the salient channel configuration of the input tile, weight tiles are preloaded accordingly, and the control unit propagates control signals to decoders. **(3) Matrix Multiplication:** Data of input/output channels flows in the corresponding PE array columns/rows, respectively. The Multiply-And-Accumulate (MAC) operations for normal and outlier values can be concurrently and seamlessly executed on-chip, since both normal and outlier input values utilize the same quantization scale, and their multiplication results are stored in a unified integer format. In this sense, `Oltron` does not require complex control logic



(a) Efficient PE **(b)** Flexible PE

Figure 6. `Oltron` PE design.

to manage normal & outlier value computation. **(4) Writing Off-chip Memory:** Finally, the output buffer writes the accumulated output tile back to off-chip memory.

4.2 PE design

`Oltron`'s architecture utilizes a hybrid-PE design, which features the majority of PEs with a simplistic design to enhance efficiency, and the minority of PEs with a flexible design to provide adaptivity. The **Efficient PEs** (Fig. 6a) can only support signed `int4×int4` multiplication with limited accumulation bit-width (16-bits), resulting in better energy efficiency and reduced area. The **Flexible PEs** (Fig. 6b) are modified atop the efficient PE design for better flexibility. First, we augment the 4-bit multiplier to support both signed and unsigned input operands. The original 4-bit input integer register is extended with an extra sign bit to accommodate the modification. Second, to support `int×fp` multiplication, we add a shifter controlled by a 5-bit input exponent. Finally, we extend the accumulation bit-width to 32 to capture the highly dynamic range of `fp` inputs. With the above modification, each flexible PE can support `int4×fp8` (E5M2), and multiple flexible PEs can be composed to support `fp` inputs with more mantissa bits [15].

4.3 Decoder Design

`Oltron` employs a reconfigurable decoder design to support the processing of a mixture of data types, including `int4`, `fp8`, and `fp12`. Our current decoder design supports up to 12-bit `fp` precision, as we find that `fp12` can achieve comparable accuracy results to those of `fp16` on the tested LLMs. However, it is feasible to extend current design to support higher `fp` precision if necessary.

With a specified data type, the decoder accesses on-chip buffer accordingly. It loads 1 byte (two `int4`) or 2 bytes (two `fp8` or one `fp12` padded with 4 bits) when accessing on-chip buffer (Fig. 7(a)). The decoder also converts the fetched data into a unified exponent-integer pair format to simplify subsequent computation. For `int4`, the decoder simply fills the exponent fields with zeros, and the integer fields with the input values (Fig. 7(b)). For `fp`, it decodes each input value into exponent-integer pair with equation Eq. (1):

$$\mathtt{sign} \times (1 << \mathtt{mb} + \mathtt{mantissa}) << (\mathtt{exponent} - \mathtt{bias}) \quad (1)$$

where `mb` is the mantissa bit-width. To calculate the effective exponent, the constant bias is properly selected to avoid any fractions in integer fields, and overflow-proof subtraction module is adopted to avoid unexpected extremely large values. To calculate the integer, it requires 4 bits for `fp8` with $mb = 2$, or 8 bits for `fp12` with $mb = 6$ to represent the result. The converted exponent-integer pair of `fp8` can fit into the activation buffer of a single flexible unit (Fig. 7(c)); while that of `fp12` cannot. Therefore, we decompose `fp12`'s exponent-integer pair into two sub-pairs, and use two flexible PEs to collaboratively execute MAC operations (Fig. 7(d)).

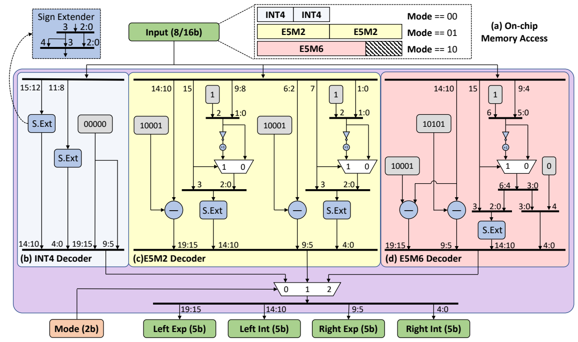


Figure 7. `Oltron` Decoder Design.

5 Evaluation

In this section, we evaluate `Oltron` in terms of model accuracy, performance, and energy efficiency.

5.1 Methodology

Quantization Setup Our method is evaluated on LLaMA(7B-65B) [16] and OPT(6.7B-66B) [24] families. We evaluate the perplexity of language generation on WikiText2 [8] and C4 [11] datasets.

Quantization Baseline We compare `Oltron` with existing weight-activation quantization works, including OmniQuant [13] and OliVe [5]. For `Oltron`, we quantize weight tensors with GPTQ [3]. All the PTQ quantization methods use 128 randomly sampled calibration sequences from WikiText2 [8].

Accelerator Baseline We compare the performance and energy of `Oltron` against two outlier-aware DNN quantization accelerators, including OLAccel [10] and OliVe [5]. We use quantized model with perplexity close to original as benchmark workload.

Architecture Implementation We implement the `Oltron` PE and decoder described in Sec. 4 with the Verilog RTL. We use Synopsys DC [7] and TSMC-28nm PDK to synthesize the designs and estimate the area, latency, and power. We use CACTI [9] to estimate the area and power of memories. We develop a cycle-level simulator based on DnnWeaver [14] to estimate the overall performance. We use DeepScaleTool [12] to scale all designs to the same process for iso-area comparison.

5.2 Accuracy Result

We first evaluate the quantized model and report the perplexity. We focus on 4-bit quantization, since near-lossless accuracy is achieved with higher bit-width in previous works [13, 20]. As shown in Tbl. 2, `Oltron` outperforms existing methods on various models. OliVe can only extend limited bit-width to encode outlier values, which leads to significant accuracy losses. OmniQuant smoothes activation distribution to encode all values with `int4`, which achieves comparable accuracy with `Oltron` on small models, but falls short in maintaining accuracy on larger models. The comparative results indicate that `Oltron`'s capability to maintain high-precision outlier quantization can effectively preserve LLMs' model accuracy. We also report the perplexity results of `Oltron` with uniform TOBE configuration across all layers. Under the same storage budget, the adaptive quantization algorithm described in Sec. 3.3 consistently outperforms the uniform counterpart.

5.3 Accelerator Performance and Power

Area We compare the accelerator area breakdown in Tbl. 3. All the accelerators use the same on-chip buffer configuration and similar core area. `Oltron` integrates more PEs within similar core area than

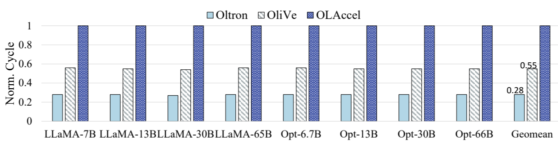
Table 2. Perplexity results of quantized model on Wikitext2 and C4 datasets (lower is better).

Model/PPL↓		LLaMA-7B		LLaMA-13B		LLaMA-30B		LLaMA-65B		OPT-6.7B		OPT-13B		OPT-30B		OPT-66B	
Method	A bits	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4	WIKI	C4
FP16	16	5.68	7.08	5.09	6.61	4.10	5.98	3.53	5.62	10.86	11.74	10.12	11.19	9.56	10.69	9.34	10.28
Olive	4	144.78	117.49	42.24	43.13	36.55	33.78	1.4e7	1.8e7	107.15	61.24	416.57	994.64	334.7	572.02	4058.83	2926.87
Omniquant	4	11.26	14.51	10.87	13.78	10.33	12.49	9.17	11.28	12.24	13.56	11.65	13.46	10.60	11.89	10.29	11.35
`Oltron`*	4.01	126.81	92.50	185.50	170.75	164.97	357.00	30.61	45.73	16.63	16.26	13.41	14.66	11.20	12.68	151.48	190.71
`Oltron`	4.01	36.47	44.62	144.08	100.18	439.25	131.15	15.85	20.85	12.69	13.58	**11.49**	**12.53**	10.72	**11.87**	11.61	11.71
`Oltron`*	4.1	14.47	16.80	**9.48**	**12.42**	**7.51**	**9.42**	**6.69**	**9.41**	**11.99**	**13.04**	**11.61**	**12.42**	10.64	**11.67**	10.50	**11.09**
`Oltron`	4.1	11.67	15.21	**8.20**	**10.84**	**6.68**	**8.65**	**5.82**	**8.19**	**12.00**	**13.02**	**11.35**	**12.27**	**10.51**	**11.63**	10.49	**11.05**

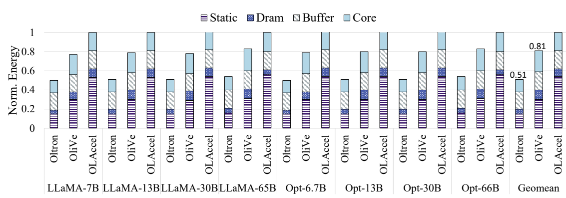
* Use the same salient channel configuration across all layers.

Table 3. The configuration and area breakdown of `Oltron` and other baselines.

Architecture	PE Number	Core Area	Buffer
`Oltron`	4096	0.322 mm^2	512 KB 4.2 mm^2
OliVe	2048	0.318 mm^2	
OLAccel	1152	0.320 mm^2	



(a) Normalized Latency



(b) Normalized Energy

Figure 8. Comparison of different accelerator designs.

OliVe, since the synthesized area of OliVe's PE is ≈ 2× larger than `Oltron`'s efficient PE design. Similar to OliVe, `Oltron`'s decoding logic placed alongside the array incurs negligible area overhead. While the complex control logic of OLAccel takes up a large portion of the total area, which constrains computational power.

Performance Fig. 8a shows the normalized latency of different designs on various models. Leveraging the higher computational power and reconfigurability, `Oltron` exhibits superior performance than the baseline designs. On average, `Oltron` achieves 1.9× and 3.6× speedup values over OliVe and OLAccel, respectively.

Energy Fig. 8b shows the normalized energy of different designs. We collect the static energy and dynamic energy (DRAM, on-chip buffer, and core). Compared with the baseline designs, `Oltron` consumes the least static energy due to better performance. `Oltron` also has lower dynamic energy of cores, owing to the simple design of the majority of PEs. Overall, `Oltron` achieves 1.6× and 1.9× energy reduction values over OliVe and OLAccel, respectively.

6 Conclusion

In this paper, We propose `Oltron`, a holistic outlier-aware quantization framework for accelerating LLMs. The key insight is to encode outlier channels at high precision while maintaining representation regularity unaffected by non-uniform outlier channel distribution. For intra-layer heterogeneity, we propose Tile-wise Outlier-Balanced Encoding (TOBE) with consistent regular memory access and computation pattern. For inter-layer heterogeneity, we propose `Oltron`, which integrates adaptive quantization algorithm and reconfigurable hardware to accommodate each layer's precision requirement with minimum overhead. `Oltron` pushes the limit of LLM post-training quantization to a new state-of-the-art. Moreover, `Oltron`'s design surpasses existing outlier-aware accelerator, OliVe, by 1.9 × performance improvement and 1.6× energy efficiency improvement.

Acknowledgments

This work is supported by National Natural Science Foundation of China (Grant No. 62032001) and 111 Project (B18001).

References

[1] Tom Brown et al. 2020. Language models are few-shot learners. *NIPS* 33 (2020), 1877–1901.
[2] Tim Dettmers et al. 2022. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339* (2022).
[3] Elias Frantar et al. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
[4] Cong Guo et al. 2022. Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization. In *MICRO*. IEEE, 1414–1433.
[5] Cong Guo et al. 2023. OliVe: Accelerating Large Language Models via Hardware-friendly Outlier-Victim Pair Quantization. In *ISCA*. 1–15.
[6] Norman P Jouppi et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *ISCA*. 1–12.
[7] Pran Kurup et al. 2012. *Logic synthesis using Synopsys®*. Springer Science & Business Media.
[8] Stephen Merity et al. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
[9] Naveen Muralimanohar et al. 2009. CACTI 6.0: A tool to model large caches. *HP laboratories* 27 (2009), 28.
[10] Eunhyeok Park et al. 2018. Energy-efficient neural network accelerator based on outlier-aware low-precision computation. In *ISCA*. IEEE, 688–698.
[11] Colin Raffel et al. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J Mach Learn Res* 21, 1 (2020), 5485–5551.
[12] Satyabrata Sarangi et al. 2021. DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era. In *ISCAS*. IEEE, 1–5.
[13] Wenqi Shao et al. 2023. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137* (2023).
[14] Hardik Sharma et al. 2016. Dnnweaver: From high-level deep network models to fpga acceleration. In *COGARCH*.
[15] Hardik Sharma et al. 2018. Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network. In *ISCA*. IEEE, 764–775.
[16] Hugo Touvron et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
[17] Ashish Vaswani et al. 2017. Attention is all you need. *NIPS* 30 (2017).
[18] Xiuying Wei et al. 2023. Outlier Suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling. *arXiv preprint arXiv:2304.09145* (2023).
[19] Wikipedia contributors. 2022. 68–95–99.7 rule — Wikipedia, The Free Encyclopedia. [Online].
[20] Guangxuan Xiao et al. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *ICML*. PMLR, 38087–38099.
[21] Zhewei Yao et al. 2022. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *NIPS* 35 (2022), 27168–27183.
[22] Zhihang Yuan et al. 2023. RPTQ: Reorder-based Post-training Quantization for Large Language Models. *arXiv preprint arXiv:2304.01089* (2023).
[23] Ali Hadi Zadeh et al. 2020. Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference. In *MICRO*. IEEE, 811–824.
[24] Susan Zhang et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).